

WeatherDash

ESP32-Based IoT Weather Monitoring System — Project Report

Platform	Wokwi Simulator (MicroPython)
Sensor	DHT22 (Temperature & Humidity)
Display	SSD1306 128x64 I2C OLED
Cloud Service	ThingSpeak

1. Project Title

WeatherDash — ESP32-Based IoT Weather Monitoring System

2. Problem Statement

Traditional weather monitoring relies on manual instruments such as thermometers and hygrometers, which must be read by hand and offer no way to track conditions remotely or over time. This creates a need for a low-cost, automated system that continuously measures environmental conditions, displays them locally, and makes the data accessible from anywhere for monitoring and analysis.

3. Objectives

- Design a low-cost IoT-based weather monitoring system using an ESP32 microcontroller.
- Measure real-time temperature and humidity using a DHT22 sensor.
- Display live sensor readings locally on an OLED screen.
- Transmit sensor data wirelessly to a cloud platform (ThingSpeak) for remote monitoring and visualization.
- Simulate and validate the complete circuit and firmware using the Wokwi online simulator before physical deployment.

4. Components and Software Used

Hardware (simulated in Wokwi):

- ESP32 DevKit v1 microcontroller
- DHT22 Temperature and Humidity Sensor
- SSD1306 128x64 I2C OLED Display
- Connecting wires

Software / Platforms:

- Wokwi Online Simulator — circuit design and simulation
- MicroPython — firmware language
- ThingSpeak — cloud IoT platform for data logging and visualization
- ssd1306.py — MicroPython driver library for the OLED display
- urequests.py — lightweight HTTP client library for MicroPython

5. Circuit Diagram

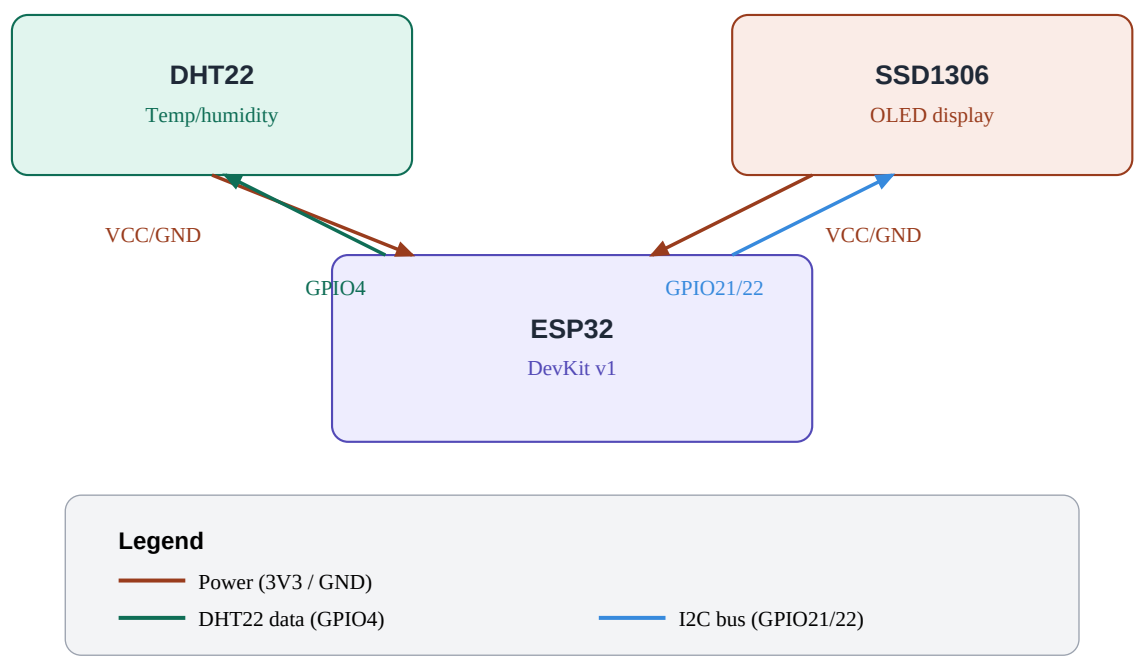


Figure 1: Wiring between the ESP32, DHT22 sensor, and SSD1306 OLED display.

Component	Pin	ESP32 Pin
DHT22	VCC	3V3
DHT22	GND	GND
DHT22	SDA (Data)	GPIO 4
OLED (SSD1306)	VCC	3V3
OLED (SSD1306)	GND	GND
OLED (SSD1306)	SDA	GPIO 21
OLED (SSD1306)	SCL	GPIO 22

6. Working Principle

The ESP32 reads temperature and humidity values from the DHT22 sensor at a regular interval (default: every 20 seconds). These readings are displayed locally on the SSD1306 OLED screen for at-a-glance monitoring, and sent over WiFi as an HTTP GET request to the ThingSpeak cloud platform, where they are logged and plotted on live charts (Field 1 = Temperature, Field 2 = Humidity). The ESP32 connects to Wokwi's built-in **Wokwi-GUEST** simulated WiFi network, which provides real internet access inside the simulation, allowing genuine communication with ThingSpeak's servers.

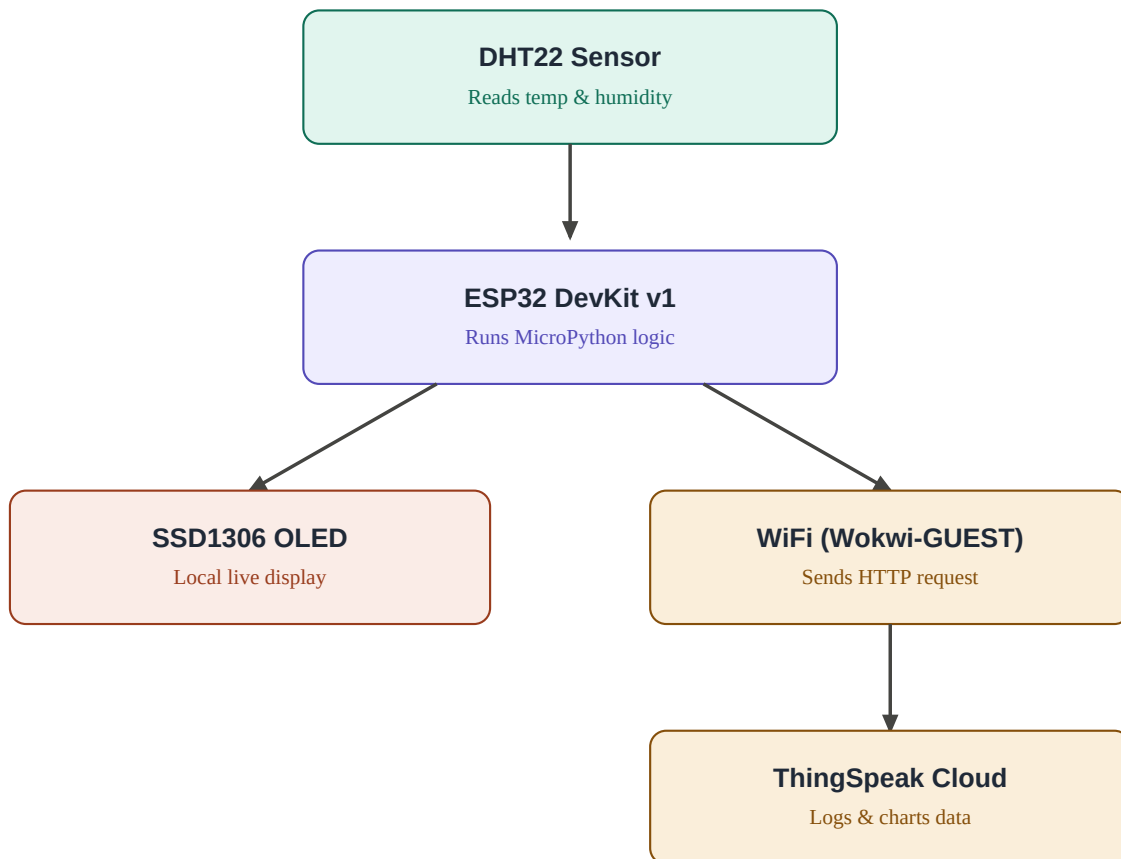


Figure 2: System-level data flow from sensor to cloud.

7. Program Explanation

The firmware (main.py) performs the following steps in a continuous loop:

- **WiFi Connection** — Connects the ESP32 to the WiFi network using the network module.
- **Sensor Reading** — Uses the dht module to trigger a measurement from the DHT22 and retrieve temperature and humidity values.
- **Local Display** — Uses the ssd1306 driver to clear the OLED buffer and write the latest readings and status to the screen.
- **Cloud Upload** — Uses the urequests module to send an HTTP GET request to ThingSpeak's update API endpoint, passing the readings as URL parameters (field1, field2) with the channel's Write API Key.
- **Error Handling** — Wraps sensor reads and network calls in try/except blocks so a failed read or upload does not crash the program; the OLED shows a status message instead.
- **Timing Control** — Waits UPDATE_INTERVAL seconds (default 20, minimum 15 per ThingSpeak's free-tier limits) before repeating the loop.

8. Output

The OLED display shows the current temperature (°C) and humidity (%), updated every cycle, along with an upload status message ("Sent OK" / "Upload failed"). The ThingSpeak channel's Private/Public View displays live line charts of temperature and humidity over time, updated automatically as new data arrives. *(Insert screenshots of your OLED output and ThingSpeak charts here.)*

9. Applications

- Home or office indoor climate monitoring
- Greenhouse and agricultural environment monitoring
- Server room / equipment room temperature-humidity logging
- Remote weather stations for field research
- Educational demonstration of IoT data pipelines (sensor → microcontroller → cloud)

10. Limitations

- The DHT22 sensor has a relatively slow response time and cannot be read more often than once every ~2 seconds.
- ThingSpeak's free tier restricts updates to a minimum of 15 seconds apart, limiting real-time responsiveness.
- The system depends on a stable WiFi connection; if the network drops, uploads fail until reconnection logic is added.
- The current implementation stores no data locally, so readings are lost if there is no internet connectivity at the time of the reading.
- Simulated in Wokwi — real-world deployment would require validation on physical hardware, including power supply stability and sensor calibration.

11. Future Scope

- Add local data logging (e.g., to an SD card) to buffer readings during internet outages, syncing once connectivity is restored.
- Add WiFi reconnection handling for resilience against dropped connections.
- Expand the system with additional sensors (e.g., air pressure, air quality, light intensity).
- Build a companion mobile app or web dashboard for real-time alerts (e.g., high humidity warnings).
- Migrate from ThingSpeak's HTTP API to MQTT for more efficient, real-time data transmission.
- Move from simulation to a physical ESP32 + DHT22 + OLED deployment.

12. Team Members' Details

Name	UUCMS No.
Denzin Xavier S	U03ZW24S0095
Manimaran I	U03ZW24S0087
Bharat C	U03ZW24S0082

13. Wokwi Project Link

<https://wokwi.com/projects/475869162909963265>

14. ThingSpeak Channel Link

<https://thingspeak.mathworks.com/channels/3504129/sharing>